Laboratorium 1 - Podstawy programowania funkcyjnego

Spis treści

Instalacja środowiska . . . 2
- Wymagania systemowe . . . 2
- Sprawdzenie obecnej wersji Python . . . 2
- Tworzenie środowiska wirtualnego . . . 2
- Instalacja narzędzi . . . 3
- Sprawdzenie instalacji . . . 3

Python i typy . . . 4
- Podstawy type hints . . . 4
- Podstawowe typy . . . 4
- Union i Optional . . . 5
- Własne typy i aliasy . . . 5
- Dataclass - klasy z danymi . . . 5
- Zadania . . . 6

Definiowanie funkcji . . . 7
- Argumenty domyślne . . . 7
- Currying i częściowe przypisane argumentów . . . 7

List comprehension . . . 9
- Podstawowa składnia . . . 9
- Filtrowanie z warunkiem . . . 9
- Zadania . . . 9

Praca z modułami . . . 10
- Tworzenie własnego modułu . . . 10
- Importowanie modułów . . . 10

Projekt końcowy . . . 11
- Zadanie: System wydatków osobistych . . . 11

Źródła . . . 12

Celem laboratorium jest wprowadzenie do podstawowych koncepcji programowania funkcyjnego z wykorzystaniem języka Python. Podczas zajęć poznasz typy danych, definiowanie funkcji, list comprehensions oraz organizację kodu w moduły. Na końcu zaimplementujesz prosty system śledzenia wydatków osobistych.

Instalacja środowiska

Wymagania systemowe

Do laboratorium potrzebne są:

- Python 3.12 lub nowszy (ze względu na lepsze wsparcie typów)
- pip (menedżer pakietów Python)
- mypy (narzędzie do statycznej analizy typów)

Sprawdzenie obecnej wersji Python

Sprawdź czy masz zainstalowany Python w odpowiedniej wersji:

Windows

```
python --version
```

macOS/Linux

```
python3 --version
```

Jeśli wersja jest starsza niż 3.12, pobierz i zainstaluj najnowszą wersję ze strony https://www.python.org/downloads/

Tworzenie środowiska wirtualnego

Środowisko wirtualne izoluje zależności projektu od systemu. Utwórz je następująco:

Windows

```
cd lab1
python -m venv venv
venv\Scripts\activate
```

macOS/Linux

```
cd lab1
python3 -m venv venv
source venv/bin/activate
```

Po aktywacji wiersz poleceń powinien pokazywać `(venv)` na początku.

Instalacja narzędzi

Zainstaluj wymagane narzędzia do pracy z typami:

```
pip install mypy ①
```

① Narzędzie do statycznej analizy typów

Sprawdzenie instalacji

Sprawdź czy wszystko działa poprawnie:

```
python --version
mypy --version
```

UWAGA: Aby dezaktywować środowisko wirtualne, użyj komendy `deactivate`.

Python i typy

Python jest dynamicznie typowanym językiem, z wsparciem dla opcjonalnych typów (*ang. type hints*). Opisane typy nie są sprawdzane w czasie działania programu, ale mogą być sprawdzane z wykorzystaniem narzędzi do statycznej analizy kodu. Przykładem, a zarazem najbardziej standardowym takim narzędziem jest `mypy`.

Podstawy type hints

Utwórz plik `intro.py` z poniższymi przykładami:

```
t: int = 0.1 ①
```

① Błąd: `float` przypisany do `int`

Sprawdź z `mypy`:

```
mypy intro.py
```

Wynik:

```
error: Incompatible types in assignment (expression has type "float", variable has
type "int")  [assignment]
```

Podstawowe typy

Python oferuje wiele wbudowanych typów, które można używać w type hints:

```
①
natural: int = 42
real: float = 3.14
text: str = "Hello World"
truth: bool = True

②
list_of_number: list[int] = [1, 2, 3, 4, 5]
mixed_typle: tuple[int, str, float] = (1, "hello", 3.14)
dictionary: dict[str, int] = {"apple": 5, "banana": 3}
set_of_strings: set[str] = {"a", "b", "c"}
```

① Typy prymitywne

② Kolekcje

Union i Optional

`Union` pozwala na typy, które mogą być jednym z kilku typów:

```
from typing import Union, Optional

①
identifier: Union[int, str] = 123
identifier = "USER_124"

②
optional_str: Optional[str] = None
optional_str = "just a string"
```

① Zmienna może być `int` lub `str`

② `Optional[T]` to skrót od `Union[T, None]`

Własne typy i aliasy

Można tworzyć własne aliasy typów dla lepszej czytelności:

```
from typing import NewType, TypeAlias

type Temperatura = float ①
type Współrzędna = tuple[float, float]

UserID = NewType('UserID', int) ②
uid: UserID = UserID(12345)
```

① Nowa nazwa dla istniejącego typu

② Nowy typ oparty na istniejącym

Dataclass - klasy z danymi

Dataclass to dekorator wprowadzony w Python 3.7, który automatycznie generuje metody specjalne dla klas przechowujących dane. Jest szczególnie użyteczny w programowaniu funkcyjnym, gdzie często operujemy na strukturach danych.

```
from dataclasses import dataclass
from typing import Optional

@dataclass ①
class Person:
    name: str
    age: int
    email: Optional[str] = None ②
```

```
@dataclass(frozen=True) ③
class Point:
    x: float
    y: float

    def distance_from_origin(self) -> float:
        return (self.x ** 2 + self.y ** 2) ** 0.5
```

① Automatycznie generuje metody niezbędne do pracy z strukturą

② Pole opcjonalne z wartością domyślną

③ `frozen=True` czyni obiekt niemodyfikowalnym

Użycie:

```
# Object initialization
person = Person("Jan Kowalski", 25, "jan@example.com")
point = Point(3.0, 4.0)

print(person)  ①
print(point.distance_from_origin())  ②

# Object comparison
person1 = Person("Anna", 30)
person2 = Person("Anna", 30)
print(person1 == person2)  ③
```

① Wyświetli: `Person(name='Jan Kowalski', age=25, email='jan@example.com')`

② Wyświetli: `5.0`

③ Wyświetli: `True`

Zadania

1. Napisz funkcję `validate_age(age: Union[int, str]) → bool`
    - Podpowiedź: Możesz użyć `isinstance` aby sprawdzić typ zmiennej.
2. Stwórz dataclass `Student` z polami: `name`, `age`, `grades` (lista)

Definiowanie funkcji

Python umożliwia definiowanie funkcji nazywanej z wykorzystaniem słowa kluczowego `def`. Funkcja może być zdefiniowana w module, klasie lub innej funkcji. Miejsce jej definicji wpływa na jej widoczność oraz żywotność (*ang. lifetime*).

Zdefiniujmy prostą funkcję, przyjmującą dwa argumenty `x: int` oraz `y: int`, czyli dwie liczby naturalnie oraz zwracającą jedną liczbę naturalną `int`:

```
def multiply(x: int, y: int) -> int:
    return x * y
```

UWAGA: Pomimo definicji typów, Python nie sprawdza ich w czasie działania programu.

Funkcje w Pythonie są obiektami pierwszej klasy - mogą być przypisywane do zmiennych, przekazywane jako argumenty i zwracane z innych funkcji:

```
def add(x: int, y: int) -> int:
    return x + y

def apply_operation(op: callable, a: int, b: int) -> int: ①
    return op(a, b)

result = apply_operation(add, 5, 3)
operation = add ②
```

① Funkcja jako argument (o tym więcej będzie na kolejnych zajęciach)

② Przypisywanie funkcji jako wartość zmiennej

Argumenty domyślne

Python wspiera argumenty domyślne, które upraszczają wiele przypadków:

```
def greet(name: str, prefix: str = "Hello") -> str:
    return f"{prefix}, {name}!"

print(greet("Anna"))              # Hello, Anna!
print(greet("Bob", "Hi there"))   # Hi there, Bob!
```

Currying i częściowe przypisane argumentów

Currying to technika przekształcenia funkcji przyjmującej wiele argumentów w serię funkcji przyjmujących pojedyncze argumenty. Częściowe przypisywanie argumentów (*partial application*) pozwala "zamrozić" część argumentów funkcji.

Functools.partial

Biblioteka `functools` oferuje gotowe narzędzie do częściowego przypisywania:

```
from functools import partial

def power(base: int, exponent: int) -> int:
    return base ** exponent

square = partial(power, exponent=2) ①

print(square(5))  # 25
```

① Ustawienie potęgi jako 2

Currying manualny

Można też implementować currying "ręcznie":

```
def curry_add(x: int):
    def inner(y: int) -> int:
        return x + y
    return inner

add_five = curry_add(5)
result = add_five(3) # result = 8
```

List comprehension

List comprehension to zwięzły sposób tworzenia list w Pythonie. Pozwala na tworzenie nowych list poprzez zastosowanie wyrażeń do wszystkich elementów w kolekcji.

Podstawowa składnia

```
numbers = [1, 2, 3, 4, 5]
squares = [x**2 for x in numbers] ①
print(squares)  # [1, 4, 9, 16, 25]

words = ["hello", "world", "python"]
uppercase = [word.upper() for word in words] ②
print(uppercase)  # ['HELLO', 'WORLD', 'PYTHON']
```

① Podnoszenie każdego elementu do kwadratu

② Zamiana każdego słowa na wielkie litery

Filtrowanie z warunkiem

List comprehension może zawierać warunek filtrujący:

```
numbers = range(1, 11)
even_squares = [x**2 for x in numbers if x % 2 == 0] ①
print(even_squares)  # [4, 16, 36, 64, 100]

names = ["Anna", "Bob", "Charlie", "Diana"]
short_names = [name for name in names if len(name) <= 3] ②
print(short_names)  # ['Bob']
```

① Kwadraty tylko parzystych liczb

② Imiona krótsze niż 4 znaki

Zadania

Wykorzystaj list comprehension do rozwiązania następujących problemów:

1. Mając listę liczb `[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]`, stwórz listę kwadratów liczb parzystych.
   - Oczekiwany wynik: `[4, 16, 36, 64, 100]`
2. Mając listę słów `["python", "java", "rust", "go", "c"]`, stwórz listę długości każdego słowa, ale tylko dla słów dłuższych niż 2 znaki.
   - Oczekiwany wynik: `[6, 4, 4]`

Praca z modułami

Modularność to kluczowa cecha programowania funkcyjnego. Python pozwala na organizację kodu w moduły, które można importować i używać w innych częściach programu.

Tworzenie własnego modułu

Utwórz plik `math_utils.py`:

```
def factorial(n: int) -> int:
    """Calculates factorial of N."""
    if n <= 1:
        return 1
    return n * factorial(n - 1)

def fibonacci(n: int) -> int:
    """Calculates n-th fibbonacci number recursively."""
    if n <= 1:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)

PI: float = 3.14159 # Constant
```

Importowanie modułów

Istnieją różne sposoby importowania tego samego modułu:

```
import math_utils ①
result = math_utils.factorial(5)

from math_utils import factorial, PI ②
result = factorial(5)
pi_value = PI

from math_utils import fibonacci as fib ③
result = fib(10)

import math_utils as math ④
result = math.factorial(5)
```

① Import całego modułu

② Import konkretnych elementów

③ Import z aliasem

④ Import modułu z aliasem

Projekt końcowy

Zadanie: System wydatków osobistych

Twoim zadaniem jest implementacja prostego systemu do śledzenia wydatków osobistych z wykorzystaniem poznanych elementów programowania funkcyjnego.

W ramach pracy nad projektem, wypełnić należy nie-zaimplementowane fragmenty kodu w pliku `expenses.py`. Fragmenty te oznaczone są komentarzem `TODO`.

Testowanie

Uruchomienie pliku `main.py` testuje wszystkie funkcje:

```
mypy main.py ①
python main.py  ②
```

① Aby sprawdzić poprawność type hint-ów

② Aby sprawdzić poprawność działania kodu

Kryteria oceny

- **Type hints** (20%) - użyte wartości są zgodne z type hint-ami
- **Dataclass** (20%) - implementacja Transaction z odpowiednimi polami
- **Implementacja funkcji** (50%) - funkcje działają zgodnie ze specyfikacją
- **Organizacja kodu** (10%) - Dobre formatowanie, czytelny kod
- **Funkcje dodatkowe** (do 15% bonusu) - za `create_category_filter` i `create_amount_filter`.

Źródła

- https://docs.python.org/3/library/typing.html
- https://docs.python.org/3/library/functools.html